

# GUION DE EXPOSICIÓN CON ACCIONES EN PANTALLA

Segundo Parcial - Automatización web y API

Duración objetivo: 9:30 a 10:00 minutos

Selenium + JUnit | RestAssured + JUnit | Page Object Model

Cada sección indica qué decir y qué mostrar en IntelliJ.

# Preparación antes de exponer

Haz esto unos minutos antes de compartir tu pantalla:

| Paso | Preparación         | Acción                                                                                          |
|------|---------------------|-------------------------------------------------------------------------------------------------|
| 1    | Abrir IntelliJ      | Abrir el proyecto automatizacion-saucedemo-api.                                                 |
| 2    | Expandir el árbol   | Abrir src > test > java > com.ramiro.automation.                                                |
| 3    | Preparar pestañas   | Abrir pom.xml, BasePage, InventoryPage, BaseWebTest, SauceDemoWebTests y RestfulBookerApiTests. |
| 4    | Cerrar ruido        | Cerrar Problems si está abierto. Los indicadores verdes de ortografía no afectan las pruebas.   |
| 5    | Comprobar Internet  | Sauce Demo y Restful Booker son servicios externos.                                             |
| 6    | No ejecutar todavía | Dejar SauceDemoWebTests visible para iniciar la explicación con calma.                          |

Regla sencilla: primero explica el archivo, después señala el código y recién entonces cambia de pestaña.

# Guion guiado

| 0:00-0:45                                                                                                                                                                                                                                                                                                                                                                                                                              | 1. Presentación                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>Buenos días. Mi proyecto del segundo parcial consiste en automatizar casos de prueba que primero fueron documentados en TestLink. Seleccioné cinco casos web de Sauce Demo y cinco casos de la API Restful Booker. La parte web está desarrollada con Selenium WebDriver y JUnit 5. La parte API utiliza RestAssured y JUnit 5. Todo está programado en Java 17 y organizado como un proyecto Maven.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>Muestra el nombre del proyecto en la parte superior. No abras archivos todavía. Señala brevemente las carpetas api, pages y web.</p> |

Señal para continuar: Cuando termines de nombrar las herramientas, abre SauceDemoWebTests.

| 0:45-1:35                                                                                                                                                                                                                                                                                                                                                                                                                             | 2. Relación con TestLink                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>Estos casos no fueron inventados ni reenumerados. Conservé los códigos reales de TestLink. Para web seleccioné WEB-05, WEB-06, WEB-07, WEB-09 y WEB-20. Para API seleccioné API-01 hasta API-05, porque esos son sus identificadores reales. En TestLink estaban escritos los pasos y resultados esperados. En este proyecto los resultados esperados fueron convertidos en assertions automáticas.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>En SauceDemoWebTests, señala las anotaciones @DisplayName donde aparecen WEB-05, WEB-06 y WEB-07. Desplázate un poco para mostrar WEB-20 y WEB-09. No ejecutes todavía.</p> |

Señal para continuar: Vuelve al inicio del archivo y abre pom.xml.

| 1:35-2:25                                                                                                                                                                                                                                                                                                                                                                                        | 3. Herramientas y Maven                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>JUnit es el framework que reconoce los métodos marcados con arroba Test, los ejecuta y muestra cuáles aprobaron o fallaron. Selenium controla Edge para interactuar con la página. RestAssured envía solicitudes HTTP directamente a la API. Maven descarga las dependencias, compila y ejecuta las pruebas. Jackson ayuda a convertir los datos Java en JSON.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>Abre pom.xml. Señala, sin leer cada línea, las dependencias selenium-java, junit-jupiter, rest-assured y jackson-databind. Señala también Java 17 y la propiedad headless.</p> |

Señal para continuar: Cierra pom.xml o cambia al árbol del proyecto.

| 2:25-3:25                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 4. Organización del código                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>El proyecto está dividido en tres paquetes. API contiene las cinco pruebas de Restful Booker. Web contiene la preparación del navegador y las cinco pruebas de Sauce Demo. Pages contiene el patrón Page Object Model. En pages, cada clase representa una pantalla: LoginPage para el acceso, InventoryPage para productos, CartPage para el carrito, ProductDetailsPage para el detalle y las clases Checkout para las etapas de compra.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>En el árbol, expande api, pages y web. Pasa el cursor lentamente por RestfulBookerApiTests, BasePage, InventoryPage, BaseWebTest y SauceDemoWebTests. No abras target: es una carpeta generada por Maven.</p> |

Señal para continuar: Abre BasePage.

| 3:25-4:35                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 5. Page Object Model                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>Page Object Model separa las acciones de las páginas de los casos de prueba. BasePage contiene métodos compartidos para esperar elementos, hacer clic, escribir y leer texto. InventoryPage conoce los selectores de la pantalla de productos y ofrece métodos como agregar Backpack o abrir el carrito. El test solo llama esos métodos y luego realiza sus assertions. Si cambia un selector, se modifica una página y no todas las pruebas.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>En BasePage, señala los métodos visible, click, type y text. Después abre InventoryPage y señala un selector por id y el método addBackpackToCart.</p> |

Señal para continuar: Antes de cambiar, aclara: Page Object Model no es lo mismo que pom.xml de Maven.

| 4:35-5:55                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 6. Cinco pruebas web                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>WEB-05 ordena los productos por precio y valida el primer y último producto. WEB-06 agrega Backpack y comprueba que el contador sea uno. WEB-07 elimina el producto y confirma que el carrito quede vacío. WEB-20 abre el detalle de Backpack y valida nombre, precio, descripción y botón. WEB-09 completa una compra y comprueba el mensaje final. Login y Logout no son casos de prueba. LoginPage se utiliza solamente como preparación y no contiene arroba Test.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>Abre SauceDemoWebTests. Señala cada @Test y una assertion: assertEquals o assertTrue. Cuando menciones que Login no cuenta, muestra que los cinco métodos son los casos WEB seleccionados.</p> |

Señal para continuar: Después abre BaseWebTest.

| 5:55-6:50                                                                                                                                                                                                                                                                                                                                                                           | 7. Independencia de las pruebas web                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>Cada caso web debe ser independiente. Antes de cada prueba, BaseWebTest crea Edge, abre Sauce Demo, limpia localStorage, sessionStorage y cookies, recarga la página e inicia sesión como preparación. Después de cada prueba, driver quit cierra el navegador. Esto evita que un carrito o una sesión anterior provoquen resultados incorrectos.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>En BaseWebTest, señala @BeforeEach, la limpieza de almacenamiento, el login de preparación y @AfterEach. Señala driver.quit al final.</p> |

Señal para continuar: Abre RestfulBookerApiTests.

| 6:50-8:05                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 8. Cinco pruebas API                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>Las pruebas API utilizan el estilo given, when y then. Given prepara headers y JSON. When ejecuta GET o POST. Then contiene las assertions. API-01 crea un token con POST auth. API-02 obtiene IDs con GET booking. API-03 crea una reserva con POST booking. API-04 consulta la reserva creada mediante GET y su identificador. API-05 consulta un identificador inexistente y espera 404. Así se cumplen Rest calls, assertions, GET y POST.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>En RestfulBookerApiTests, señala baseUrl. Después muestra un bloque given / when / then. Señala .post, .get, statusCode(200), una validación body y finalmente statusCode(404).</p> |

Señal para continuar: Regresa a SauceDemoWebTests para iniciar la demostración.

| 8:05-9:15                                                                                                                                                                                                                                                                                                                                                                                                      | 9. Ejecución en IntelliJ                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>LO QUE DICES</b></p> <p>Para ejecutar los casos web abro SauceDemoWebTests y pulso el triángulo verde de la clase. Edge puede abrirse de manera visible cuando headless está desactivado. Para API hago lo mismo en RestfulBookerApiTests, pero no se abre navegador porque RestAssured se comunica directamente con el servidor. También puedo ejecutar las diez pruebas con Maven usando mvn test.</p> | <p><b>ACCIÓN EN PANTALLA</b></p> <p>Pulsa el triángulo verde junto a SauceDemoWebTests y elige Run. Mientras Edge trabaja, sigue explicando. Cuando termine, señala 5 tests passed. Luego ejecuta RestfulBookerApiTests y señala sus cinco resultados. Si el tiempo es corto, muestra el resultado anterior sin repetir ambas ejecuciones.</p> |

Señal para continuar: Deja visible el panel Run con resultados verdes.

9:15-10:00

## 10. Resultado y cierre

## LO QUE DICES

La ejecución final completó diez pruebas: cinco web y cinco API, con cero fallos y cero errores. El código está publicado en GitHub, en el repositorio SegundoParcial. Con este trabajo convertí casos manuales de TestLink en pruebas automatizadas, independientes y mantenibles utilizando selectores, assertions, Page Object Model, Rest calls, GET, POST y JUnit. Gracias.

## ACCIÓN EN PANTALLA

Señala el resumen verde del panel Run. Después vuelve al árbol para que se vean api, pages y web. Termina sin seguir navegando ni modificar código.

Señal para continuar: Haz una pausa y espera preguntas.

## Plan de emergencia durante la demostración

| Pregunta                   | Respuesta corta                                                                            |
|----------------------------|--------------------------------------------------------------------------------------------|
| Edge no aparece            | Explica que headless está activo. La prueba sigue ejecutándose sin ventana.                |
| La web tarda               | No reinicies. Continúa explicando Page Object Model mientras termina.                      |
| La API tarda               | Menciona que depende de un servicio externo y muestra una ejecución anterior.              |
| Aparecen palabras verdes   | Son sugerencias del corrector ortográfico para español y nombres técnicos; no son errores. |
| Te preguntan por LoginPage | Responde que es una precondición, no un @Test ni uno de los cinco casos.                   |
| Te pierdes en el código    | Vuelve al árbol, abre SauceDemoWebTests y retoma desde los códigos WEB.                    |
| Quedan menos de 2 minutos  | Salta directamente a los resultados y lee el bloque final del guion.                       |

No modifiques código durante la exposición. Si una prueba externa falla por conexión, explica el resultado esperado y muestra el reporte anterior.

## Posibles preguntas del docente

| Pregunta                      | Respuesta corta                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------|
| ¿Qué es JUnit?                | Es el framework que ejecuta los métodos @Test y reporta aprobados o fallados.        |
| ¿Qué es Page Object Model?    | Un patrón que separa selectores y acciones de pantalla de los tests.                 |
| ¿POM y pom.xml son lo mismo?  | No. Page Object Model es el patrón; pom.xml es la configuración Maven.               |
| ¿Por qué existe LoginPage?    | Se usa como preparación para llegar al inventario; no es un caso @Test.              |
| ¿Probaste Login o Logout?     | No. La consigna prohíbe repetirlos como escenarios.                                  |
| ¿Por qué Selenium?            | Porque permite automatizar la interacción real con Edge y la interfaz web.           |
| ¿Por qué RestAssured?         | Porque permite construir requests y validar respuestas HTTP y JSON.                  |
| ¿Qué es una assertion?        | Una comparación automática entre el resultado esperado y el real.                    |
| ¿Qué selectores usaste?       | Principalmente id y CSS, porque son claros y estables.                               |
| ¿Qué significa GET?           | Consulta información sin crear un recurso nuevo.                                     |
| ¿Qué significa POST?          | Envía datos para autenticar o crear un recurso.                                      |
| ¿Qué significa HTTP 200?      | La solicitud fue procesada correctamente.                                            |
| ¿Qué significa HTTP 404?      | El recurso consultado no existe.                                                     |
| ¿Por qué API no abre Edge?    | RestAssured se comunica directamente con el servidor mediante HTTP.                  |
| ¿Cómo aseguras independencia? | Cada prueba web limpia estado, crea navegador y lo cierra; la API prepara sus datos. |
| ¿Qué hace Maven?              | Descarga dependencias, compila y ejecuta las pruebas.                                |
| ¿Qué es headless?             | Ejecutar el navegador sin mostrar su ventana.                                        |
| ¿Los códigos son inventados?  | No. WEB y API conservan los identificadores reales de TestLink.                      |
| ¿Qué resultado obtuviste?     | Diez pruebas ejecutadas, cero fallos y cero errores.                                 |
| ¿Dónde está el proyecto?      | En GitHub: Ramiro-sakv/SegundoParcial.                                               |



## Tarjeta rápida para tener al lado

Orden de pestañas: SauceDemoWebTests → pom.xml → BasePage → InventoryPage → BaseWebTest → RestfulBookerApiTests → Run.

| # | Tema       | Frase clave                        |
|---|------------|------------------------------------|
| 1 | 10 pruebas | 5 web + 5 API                      |
| 2 | Web        | Selenium + JUnit                   |
| 3 | API        | RestAssured + JUnit                |
| 4 | Patrón     | Page Object Model                  |
| 5 | Requests   | GET y POST                         |
| 6 | Validación | Assertions, statusCode y body      |
| 7 | Login      | Solo preparación, no caso          |
| 8 | Resultado  | 10 ejecutadas, 0 fallos, 0 errores |

Última frase: Convertí casos manuales de TestLink en pruebas automatizadas, independientes y mantenibles con JUnit.